Diagramas de domínio: Oficina Mecânica

Tech Challenge SOAT, fase 1, grupo 31

Domain Storytelling feito pelo grupo no egon.io (25/09/2026) e diagramas complementares de DDD (agregados, entidades, objetos de valor, contextos delimitados e máquinas de estado) montados a partir do board de Event Storming no Miro e conferidos com o código.

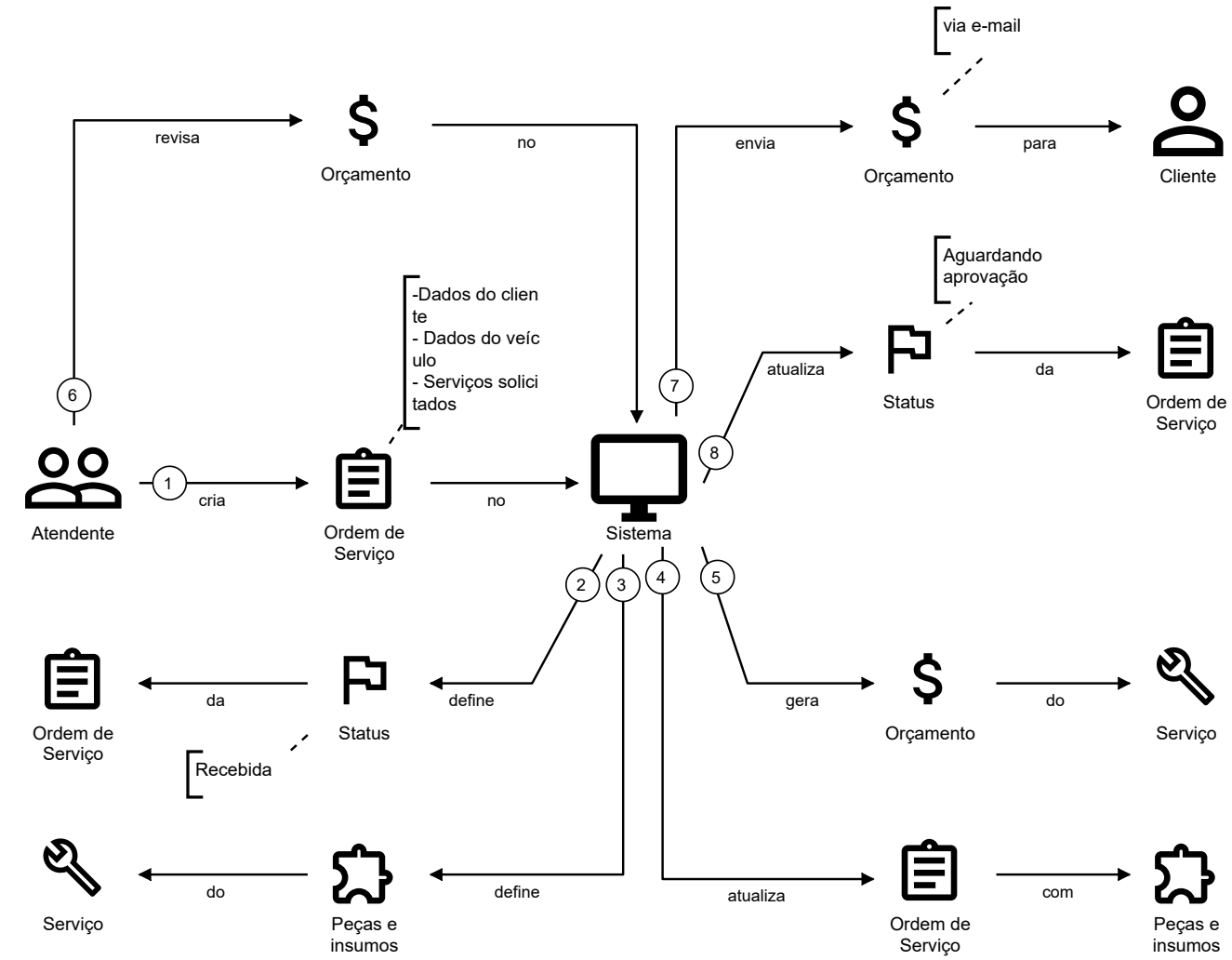
Conteúdo

Página	Diagrama	Origem
2	1. Criação da OS com serviço	egon.io (`docs/ddd/egon/1-criacao-os-com-servico.egn.svg`)
3	2. Criação da OS com diagnóstico	egon.io
4	3. Acompanhamento com aprovação total	egon.io
5	4. Acompanhamento com aprovação parcial	egon.io
6	5. Acompanhamento com reprovação	egon.io
7	6. Gerenciamento de peças e insumos	egon.io
8	Agregados, entidades e objetos de valor	Miro (agregados) + código
9	Contextos delimitados e mapa de contextos	Miro + código
10	Máquinas de estado da OS e do orçamento	código

Cada história traz, ao lado, como o código atual implementa o fluxo. Onde a história e o código divergem, a divergência está indicada e também listada em `docs/ddd/linguagem-ubiqua.md` (seção "Relação com o código"). O board do Miro: https://miro.com/app/board/uXjVHnerRBQ=/

1. Criação da OS com serviço

Fluxo quando o cliente já sabe quais serviços quer (ex.: troca de óleo).



História

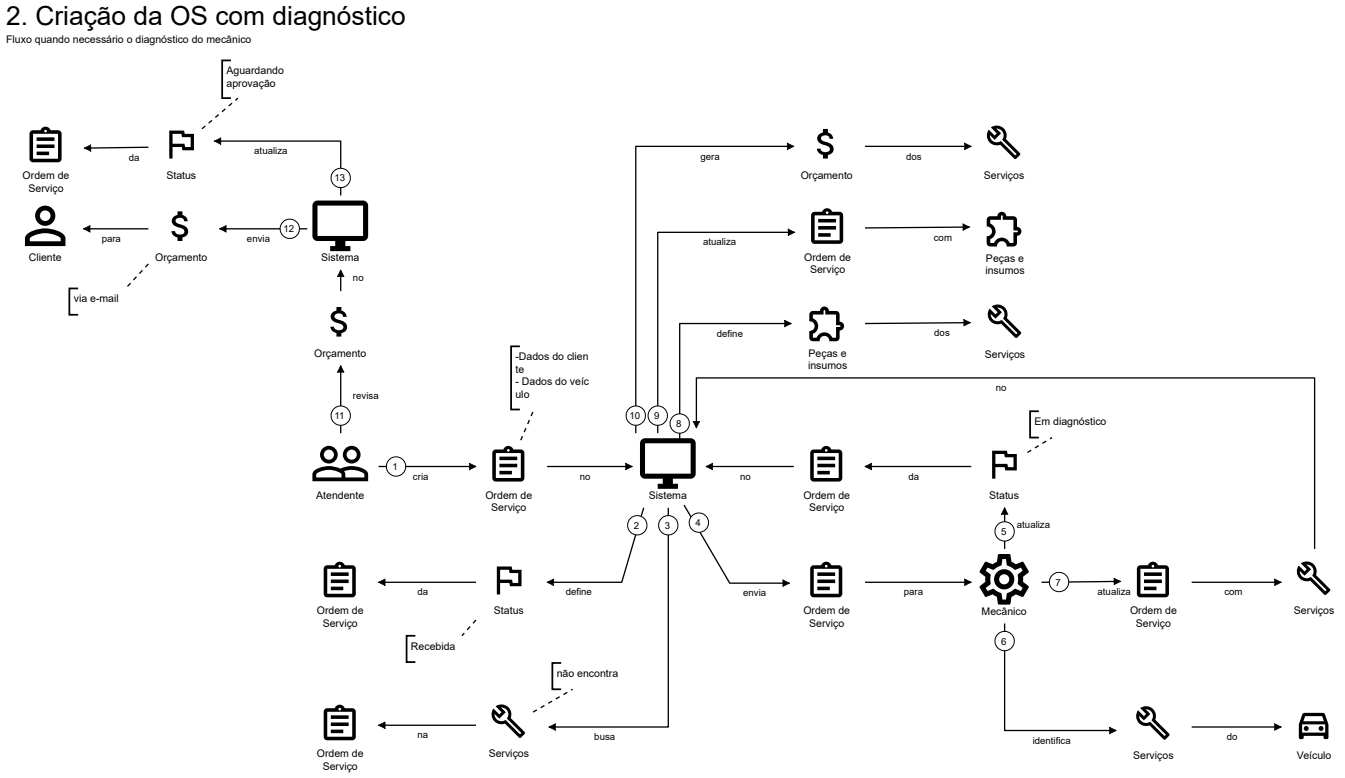
1. O atendente cria a OS com os dados do cliente, do veículo e os serviços pedidos.
2. O sistema define o status Recebida.
3. O atendente define serviços, peças e insumos; o sistema gera o orçamento.
4. O atendente revisa e o sistema envia o orçamento por e-mail; status Aguardando aprovação.

No código

- `POST /api/ordens-servico` abre a OS em `RECEBIDA` com cliente, veículo e relato; publica `OrdemServicoCriada` e o cliente recebe o código de acesso por e-mail (simulado em log).
- `POST /api/orcamentos` recebe serviços do catálogo e peças do estoque, reserva as peças e calcula o total. A OS passa por `EM_DIAGNOSTICO` e vai para `AGUARDANDO_APROVACAO` na mesma operação.
- Não existe etapa separada de "revisar"; o orçamento é enviado ao ser gerado (`OrcamentoGerado`).

2. Criação da OS com diagnóstico

Fluxo quando o problema precisa ser identificado antes do orçamento.



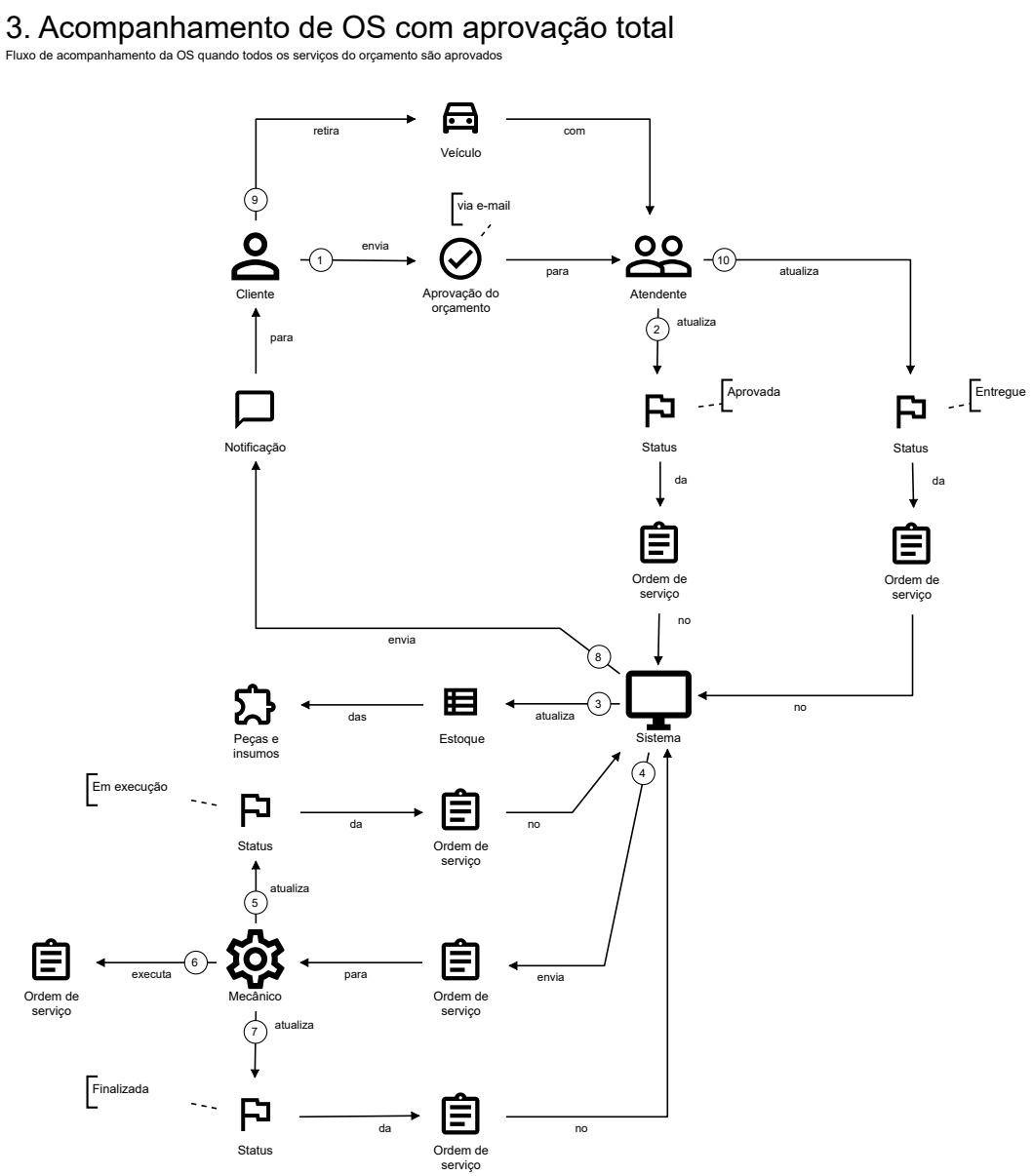
História

1. O atendente cria a OS com cliente e veículo; o sistema não encontra serviços e define Recebida.
2. A OS segue para o mecânico, que identifica os serviços (Em diagnóstico).
3. Os serviços, peças e insumos entram na OS e o sistema gera o orçamento.
4. O atendente revisa e envia por e-mail; status Aguardando aprovação.

> **No código**
>
> - O mecânico não opera o sistema (papel removido na migration V6). Ele faz o diagnóstico fora do sistema e o **atendente** registra o resultado ao gerar o orçamento.
> - O status `EM_DIAGNOSTICO` é marcado automaticamente pelo `GerarOrcamentoUseCase` quando a OS ainda está `RECEBIDA`.
> - Papéis do sistema: `ADMIN` e `ATENDENTE` (`Papel`).

3. Acompanhamento de OS com aprovação total

Todos os serviços e peças do orçamento são aprovados.



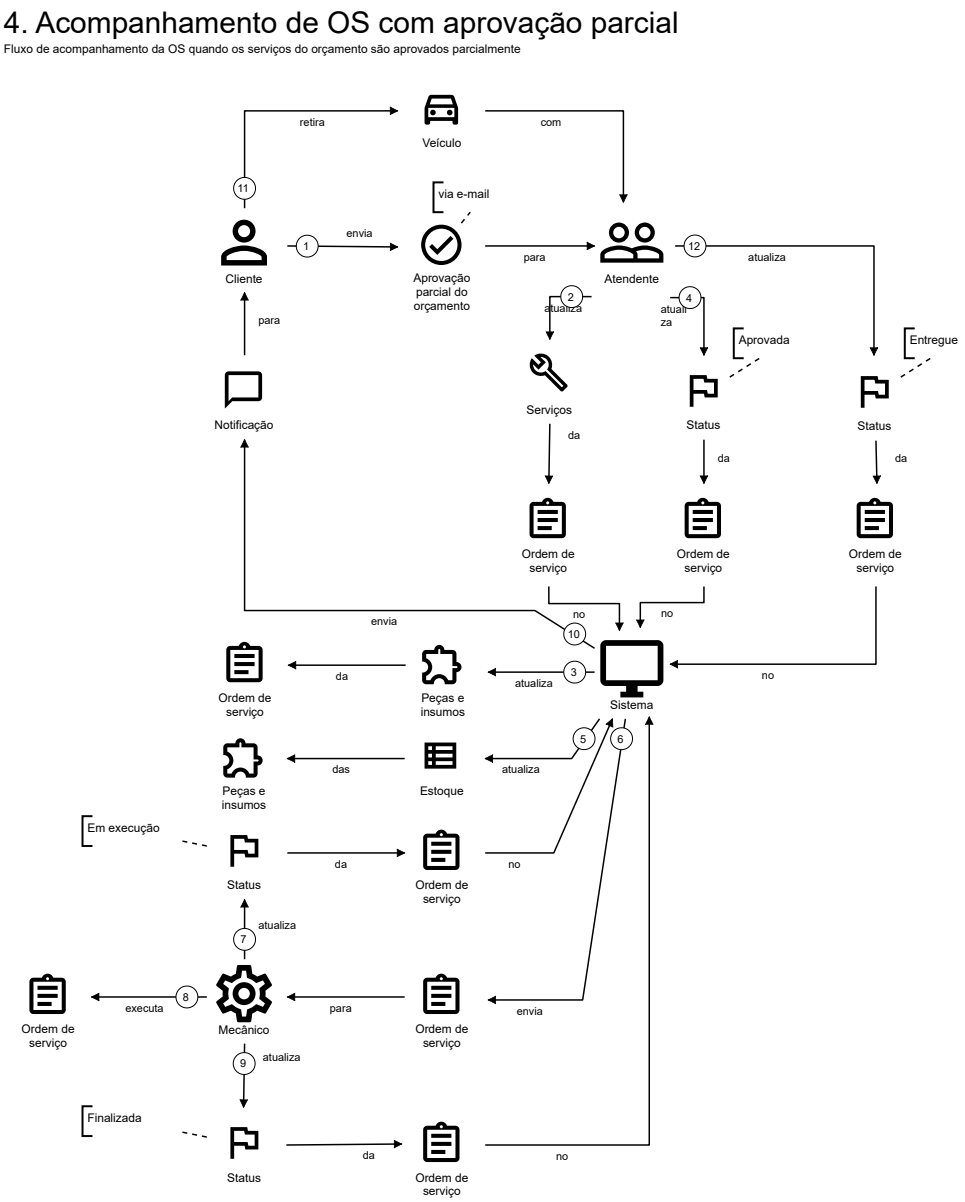
História

1. O cliente envia a aprovação do orçamento; status Aprovada e baixa de peças no estoque.
2. O mecânico executa (Em execução) e finaliza (Finalizada); o cliente é notificado.
3. O cliente retira o veículo; status Entregue.

> **No código**
>
> - Aprovação pelo cliente: `POST /api/publico/orcamentos/{id}/aprovar` com CPF/CNPJ e código de acesso; ou pelo atendente: `PATCH /api/orcamentos/{id}/aprovar`.
> - Orçamento `APROVADO`, OS `APROVADA`, reservas viram baixa (`confirmarBaixa`), evento `OrdemServicoAprovada` com e-mail simulado.
> - Execução e entrega são registradas pelo atendente: `PATCH /api/ordens-servico/{id}/iniciar-execucao`, `/finalizar` (e-mail "veículo pronto") e `/entregar`.

4. Acompanhamento de OS com aprovação parcial

O cliente aprova só parte dos serviços e peças.



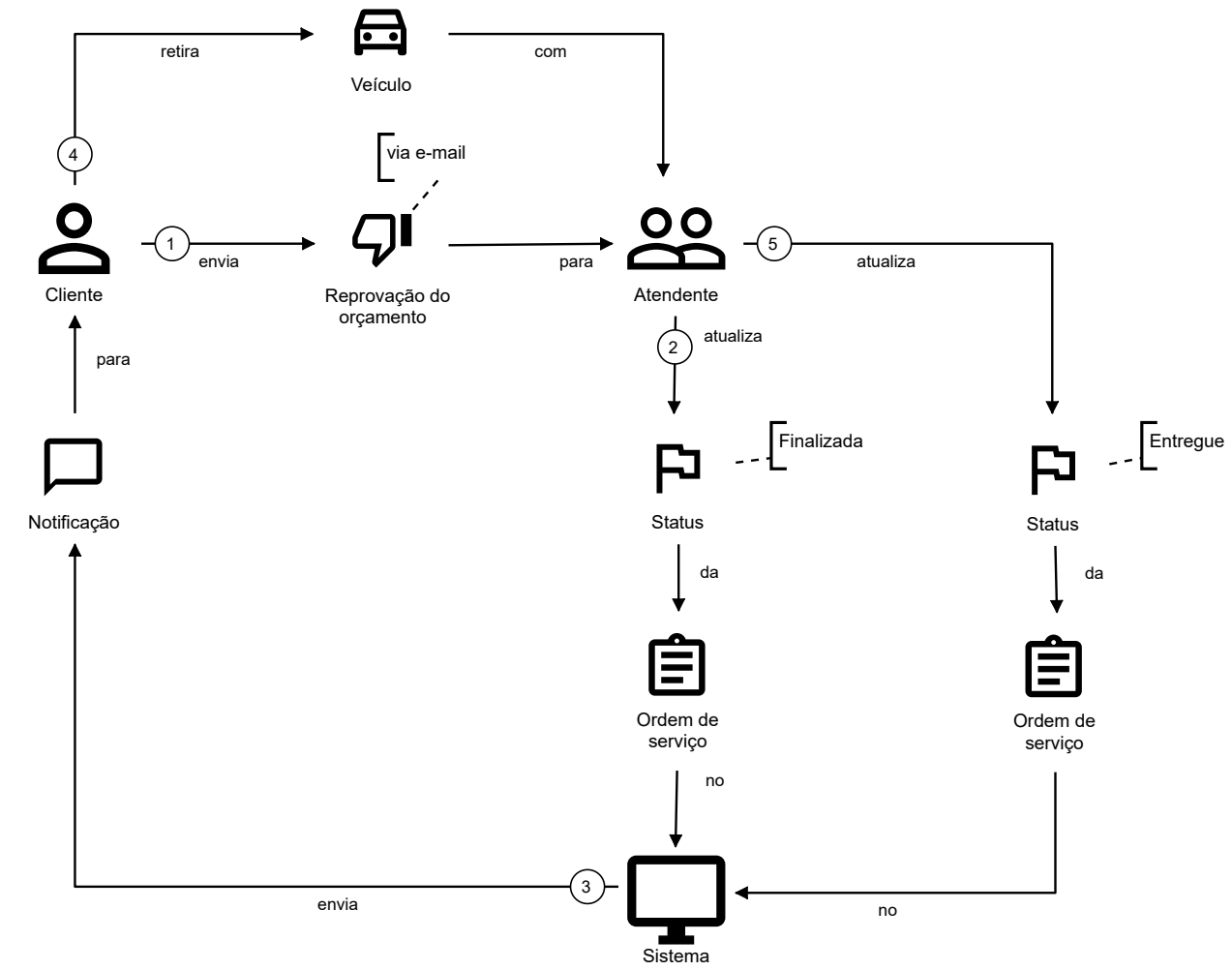
História

1. O cliente envia a aprovação parcial; a OS é atualizada só com os itens aprovados.
2. Baixa no estoque apenas das peças aprovadas; status Aprovada.
3. Execução, finalização com notificação e entrega como no fluxo 3.

> **No código**
>
> - Mesmo endpoint de aprovação, informando `servicosAprovados` e `pecasAprovadas`.
> - `Orcamento.aprovar` remove os itens não aprovados, recalcula o total e marca `APROVADO_PARCIALMENTE`.
> - Peças recusadas têm a reserva devolvida (`cancelarReserva`); as aprovadas recebem baixa.
> - Se nenhum item for aprovado, o orçamento fica `REJEITADO` e a OS `RECUSADA`.

5. Acompanhamento de OS com reprovação

O cliente reprova o orçamento.



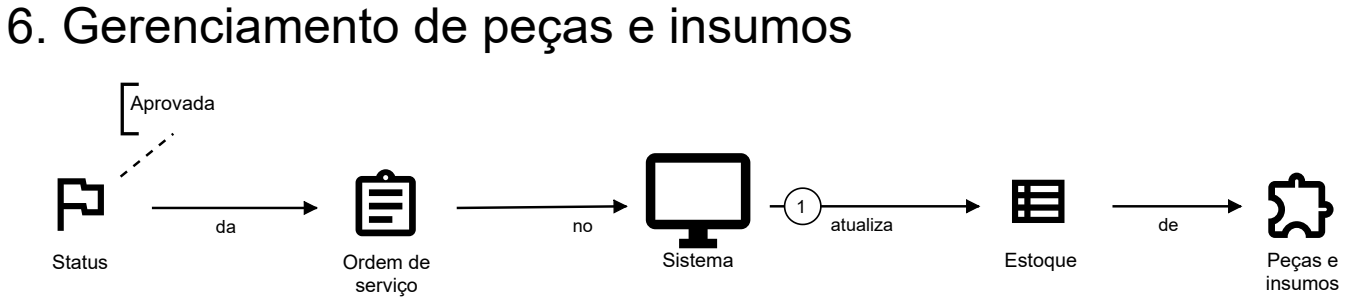
História

1. O cliente envia a reprovação; status Finalizada e notificação por e-mail.
2. O cliente retira o veículo; status Entregue.

> **No código (divergência)**
>
> - Reprovação: `POST /api/publico/orcamentos/{id}/rejeitar` (cliente) ou `PATCH /api/orcamentos/{id}/rejeitar` (atendente).
> - O orçamento fica `REJEITADO`, as reservas voltam ao estoque e a OS vai para `RECUSADA`, que é **estado final**: não passa por Finalizada nem Entregue. O cliente recebe o e-mail "orçamento não aprovado".
> - Assim, OS recusadas não entram no tempo de execução. A retirada do veículo, nesse caso, não é registrada no sistema.

6. Gerenciamento de peças e insumos

Na história do grupo: com a OS Aprovada, o sistema atualiza o estoque de peças e insumos.



Estoque dentro do fluxo da OS

- Orçamento gerado: `ProdutoEstoque.reservar` tira do disponível e soma no reservado (falha se faltar saldo).
- Orçamento aprovado: `confirmarBaixa` das peças aprovadas.
- Rejeição, aprovação parcial, expiração ou cancelamento da OS: `cancelarReserva` devolve ao disponível.

Gestão administrativa (`/api/estoque`)

- Cadastro, consulta, listagem, atualização e exclusão de peças e insumos.
- Entrada e saída manual de quantidade: `POST /{id}/adicionar` e `POST /{id}/remover`, que mexem só no disponível; reservas ficam com o orçamento.
- Nome da peça é único (restrição no banco).

Agregados, entidades e objetos de valor

No Miro o grupo separou os agregados Cadastro, Relatório, Orçamento, Estoque e Ordem de Serviço. No código, "Cadastro" virou dois agregados (Cliente e Veículo), o "Relatório" foi absorvido pelo Orçamento (o diagnóstico vira os itens) e o catálogo de serviços ganhou o seu.

Ordem de Serviço (raiz: `OrdemServico`)

Entidade raiz com `id`, cliente e veículo (por id), relato do problema, status, data de abertura e conclusão e código de acesso.
Entidades/VOs: `TransicaoStatus` (histórico de status, gravado em `historico_status_ordem_servico`), `StatusOrdemServico` (enum).
Regras: transições só pelas operações do agregado (`iniciarDiagnostico`, `anexarOrcamento`, `aprovarOrcamento`, `recusarOrcamento`, `iniciarExecucao`, `finalizar`, `entregar`, `cancelar`); relato entre 10 e 500 caracteres.
Eventos: `OrdemServicoCriada`, `OrdemServicoAprovada`, `OrdemServicoRecusada`, `OrdemServicoCancelada`, `OrdemServicoFinalizada`.

Orçamento (raiz: `Orcamento`)

Vinculado à OS por `ordemServicoId`; tem data de criação, validade e status.
VOs: `ItemServico` (serviço do catálogo, descrição, valor da mão de obra), `ItemPeca` (peça do estoque, nome, quantidade, valor unitário), `StatusOrcamento`.
Regras: só é alterado enquanto `PENDENTE`; total = mão de obra + peças; validade no futuro; expira se decidido depois da validade; aprovação total, parcial ou rejeição.
Evento: `OrcamentoGerado`.

Estoque (raiz: `ProdutoEstoque`)

Peça ou insumo com nome único, valor unitário, quantidade disponível e reservada.
Regras: reservar só com saldo disponível; baixa e cancelamento só sobre o reservado; entrada e saída manual só no disponível; valor unitário maior que zero.

Cliente (raiz: `Cliente`)

Nome, documento, e-mail, telefone.
VOs: `Endereco`, `Estado` (UF); documento validado como CPF ou CNPJ com dígitos verificadores.
Regras: documento único; não pode ser excluído com veículos vinculados.

Veículo (raiz: `Veiculo`)

Placa, marca, modelo, ano de fabricação e cliente dono (por id).
Regras: placa no padrão antigo ou Mercosul e única; não pode ser excluído com ordens de serviço vinculadas.

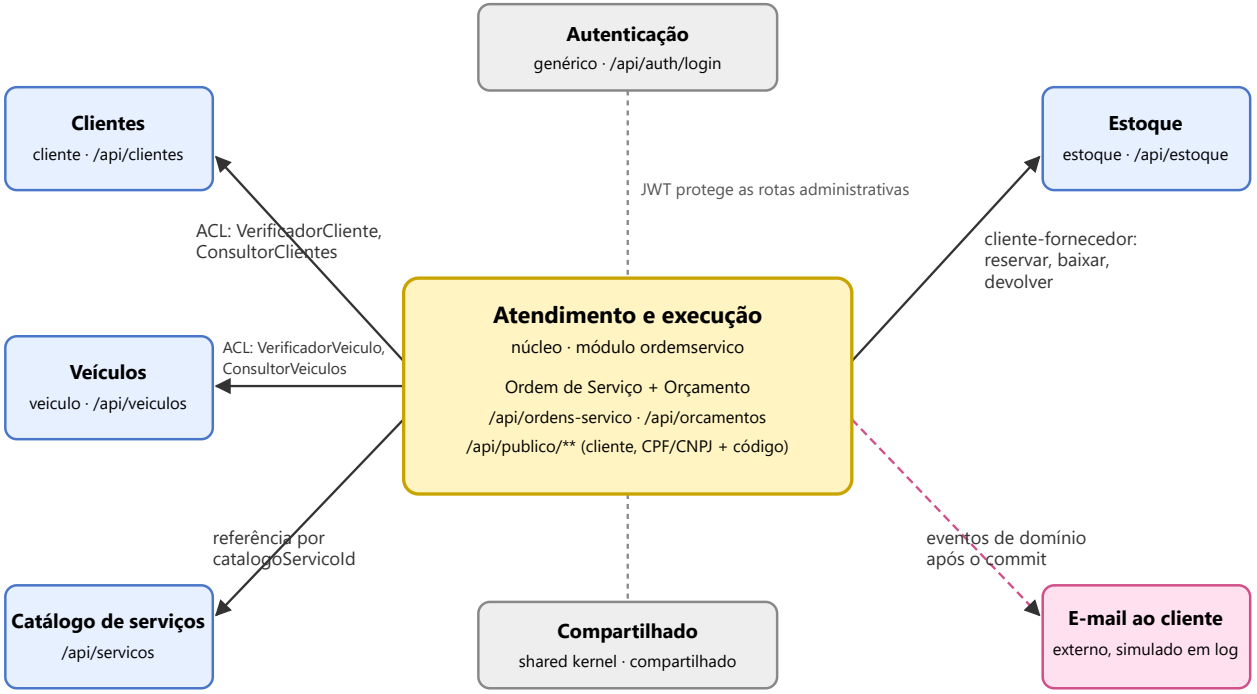
Catálogo de serviços

Serviço oferecido pela oficina (ex.: troca de óleo, alinhamento) com descrição e valor de mão de obra, mantido em `/api/servicos`. O orçamento referencia o serviço por `catalogoServicoId` e guarda uma cópia da descrição e do valor do momento.

Agregados se referenciam só por id. Usuário (`Usuario`, `Papel`) fica no contexto genérico de autenticação.

Contextos delimitados e mapa de contextos

Cada contexto é um módulo do monólito (`br.com.fiap.oficina_mecanica.*`), dividido em `domain`, `application` e `infrastructure`.



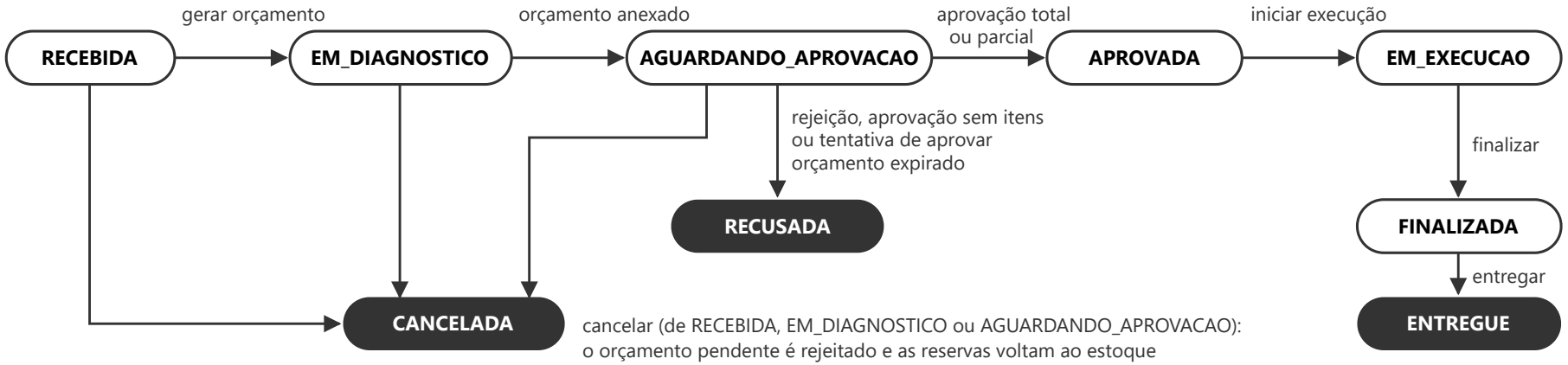
Legenda
amarelo subdomínio núcleo · azul suporte · cinza genérico/compartilhado · rosa externo

Relações

- **Clientes e Veículos → núcleo:** o núcleo depende de portas próprias (`VerificadorCliente`, `VerificadorVeiculo`, `ConsultorClientes`, `ConsultorVeiculos`) implementadas por adaptadores, que funcionam como camada anticorrupção.
- **Estoque:** o núcleo é cliente do estoque e usa o repositório de `ProdutoEstoque` na mesma transação para reservar, baixar e devolver peças.
- **Catálogo:** o orçamento guarda o `catalogoServicoId` e uma cópia da descrição e do valor.
- **Notificação:** eventos de domínio (`OrdemServicoCriada`, `OrcamentoGerado`, `OrdemServicoAprovada`, `OrdemServicoRecusada`, `OrdemServicoCancelada`, `OrdemServicoFinalizada`) são publicados depois do commit e viram e-mail simulado. Não há WhatsApp nem PDF.
- **Compartilhado:** paginação, exceções de domínio, publicação de eventos e padrão de erros, usados por todos os módulos.

Máquinas de estado

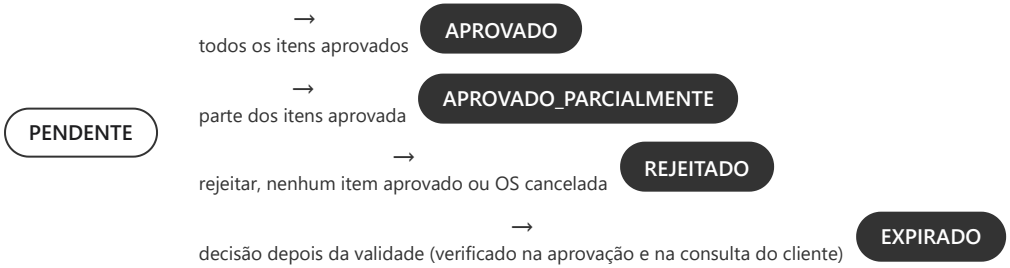
Ordem de Serviço (`StatusOrdemServico`)



Estados em preto são finais. O cliente é avisado por e-mail na criação, no orçamento gerado, na aprovação, na recusa, no cancelamento e na finalização.

Cada mudança gera uma `TransicaoStatus` (status anterior, novo e horário) no histórico. É a base do acompanhamento pelo cliente e do cálculo do tempo médio de execução (intervalo entre `EM_EXECUCAO` e `FINALIZADA`).

Orçamento (`StatusOrcamento`)



Só o orçamento `PENDENTE` aceita itens ou decisão. A expiração não depende de agendamento: é verificada no momento da ação.